2º Trabalho Computacional - Reconhecimento de Padrões: Análise de Matrizes de Covariância e Condicionamento Numérico

Lucas José Lemos Braz - 587388

[1]Programa de Pós-Graduação em Engenharia de Teleinformática (PPGETI)
Universidade Federal do Ceará (UFC)

lucas.jlb19@alu.ufc.br

1. Introdução

A caracterização estatística de classes em problemas de Reconhecimento de Padrões fundamenta-se, em grande medida, na modelagem das distribuições de probabilidade dos atributos. No caso de modelos Gaussianos, a matriz de covariância ($\mathbf{C}$) é o parâmetro que captura a dispersão dos dados e, mais criticamente, as interdependências lineares entre as variáveis. Uma estimação precisa de $\mathbf{C}$ não apenas define a geometria das fronteiras de decisão em classificadores quadráticos (CQG), mas também é a base para algoritmos de redução de dimensionalidade, como a Análise de Componentes Principais (PCA), que busca direções de máxima variância para projeção dos dados.

Este Segundo Trabalho Computacional (TC2) propõe uma investigação sobre a implementação de diferentes métodos de cálculo da matriz de covariância, utilizando como estudo de caso o dataset *Wall-Following Robot Navigation Data* do repositório UCI. Este conjunto de dados é particularmente desafiador devido à natureza das leituras de ultrassom: sensores dispostos circularmente ao redor de um robô móvel tendem a apresentar alta redundância, o que pode se traduz matematicamente em matrizes de covariância mal-condicionadas ou singulares. Analisamos aqui duas variantes do dataset (com 4 e 24 sensores) para observar como o aumento da dimensionalidade impacta a estabilidade numérica e a eficiência dos algoritmos.

Para assegurar o rigor científico e a reprodutibilidade, o projeto foi estruturado de forma modular. Toda a lógica de cálculo reside no módulo `classificao_padroes`, enquanto os experimentos e visualizações são conduzidos em notebooks Jupyter. O repositório completo pode ser acessado em: `https://github.com/LucasJLBraz/Trabalhos_Rec_Padroes`. A organização dos arquivos pertinentes a este trabalho é detalhada na Tabela **??**.

Tabela 1. Estrutura de diretórios e arquivos para o Trabalho Computacional 2.

Caminho	Descrição
`classificao_padroes/models_trabalho2.py`	Implementações manuais das matrizes de covariância.
`notebooks/Trabalho Computacional 2/`	Notebooks de experimentos e análises estatísticas.
`data/interin/`	Base de dados do robô *Wall-Following* (4 e 24 sensores).
`docs/relatorio_tc2/`	Arquivos LaTeX para a geração deste relatório técnico.

2. Metodologia

Para o desenvolvimento deste trabalho, implementamos e validamos quatro variações algorítmicas para o cálculo da matriz de covariância, fundamentais para a caracterização

estatística das classes no dataset *Wall-Following Robot Navigation*. Seja $\mathbf{X} \in \mathbb{R}^{p \times N}$ a matriz de dados, onde p é o número de sensores e N o número de amostras. A matriz de covariância sumariza as relações lineares entre os atributos, sendo definida pela esperança dos produtos externos dos desvios em relação à média aritmética de cada sensor, $\mathbf{m}$.

O primeiro método (`mcovar1`) utiliza um laço explícito para acumular os produtos externos dos desvios de cada amostra: $\mathbf{C}_1 = \frac{1}{N}\sum_{j=1}^{N}(\mathbf{x}_j - \mathbf{m})(\mathbf{x}_j - \mathbf{m})^T$. O segundo método (`mcovar2`) otimiza este cálculo via vetorização, computando $\mathbf{C}_2 = \frac{1}{N}\tilde{\mathbf{X}}\tilde{\mathbf{X}}^T$, onde $\tilde{\mathbf{X}}$ é a matriz centralizada. Embora ambos utilizem a definição direta, a abordagem matricial aproveita rotinas BLAS (*Basic Linear Algebra Subprograms*) otimizadas. O terceiro método (`mcovar3`) calcula a matriz de correlação $\mathbf{R}$ via laço e subtrai o produto externo das médias ($\mathbf{C}_3 = \mathbf{R} - \mathbf{m}\mathbf{m}^T$), enquanto o quarto método (`mcovar4`) realiza esta operação de forma puramente matricial: $\mathbf{C}_4 = \frac{1}{N}\mathbf{X}\mathbf{X}^T - \mathbf{m}\mathbf{m}^T$.

O protocolo experimental submeteu estas implementações a um *benchmark* de 100 rodadas independentes, comparando-as com a função nativa do NumPy (`np.cov`) em termos de latência média e precisão numérica. A precisão foi quantificada pela norma de Frobenius do erro residual $E = \|\mathbf{C}_{my} - \mathbf{C}_{ref}\|_F$, onde $\mathbf{C}_{ref}$ é a referência do NumPy calculada com o *bias* de divisão por N.

A estabilidade numérica foi investigada por meio da análise de posto (*rank*) e número de condicionamento (κ). Diferente de cenários onde a redundância extrema causa singularidade, buscamos avaliar se as bases de dados originais (4 e 24 sensores) são bem-condicionadas e propensas à inversão direta. Adicionalmente, aplicou-se a regularização de Tikhonov ($\mathbf{C}_{reg} = \mathbf{C} + \lambda\mathbf{I}$) de forma experimental para avaliar como a adição de um termo perturbativo na diagonal impacta o condicionamento e a invertibilidade das matrizes por classe, permitindo uma análise comparativa de robustez numérica.

3. Resultados e Discussão

A investigação empírica dos métodos manuais para estimação da matriz de covariância revelou informações relevantes sobre a eficiência da álgebra linear computacional em Python. A Tabela **??** apresenta os tempos de execução médios, evidenciando a superioridade das implementações vetorizadas.

Tabela 2. Benchmarks de tempo de execução (Média $\pm$ Desvio Padrão em ms) para 100 rodadas.

	4 Sensores		24 Sensores	
Método	**Média (ms)**	**Desvio (ms)**	**Média (ms)**	**Desvio (ms)**
`mcovar1` (Loop-Dev)	14,28	0,25	18,68	0,73
`mcovar2` (Matriz-Dev)	0,06	0,02	0,24	0,04
`mcovar3` (Loop-Mom)	14,58	0,28	19,25	0,43
`mcovar4` (Matriz-Mom)	**0,05**	0,02	**0,20**	0,03
`NumPy Cov`	0,09	0,02	0,39	0,05

As implementações matriciais (`mcovar2` e `mcovar4`) foram ordens de magnitude mais rápidas que as versões com laço ($\sim 300\times$ para 4 sensores). Notavelmente,

o método `mcovar4` superou o `mcovar2` e a própria função nativa do NumPy. A superioridade do `mcovar4` deve-se ao fato de evitar a centralização explícita da matriz, minimizando alocações de memória e maximizando o uso de rotinas BLAS otimizadas. A precisão numérica foi validada pela norma de Frobenius do erro residual (Tabela **??**), confirmando a equivalência matemática das implementações.

Tabela 3. Norma de Frobenius do erro residual ($E = \|\mathbf{C}_{my} - \mathbf{C}_{ref}\|_F$).

Método	4 Sensores	24 Sensores
Todos (C1 a C4)	$1{,}61 \times 10^{-4}$	$2{,}12 \times 10^{-3}$

3.1. Distribuição e Variabilidade dos Dados

A análise visual das distribuições dos sensores, ilustrada na Figura **??**, permite identificar como a variância se comporta entre as diferentes classes de movimento. Os histogramas e *violin plots* evidenciam que, embora existam sobreposições em certas leituras de ultrassom, classes como *Sharp-Right-Turn* apresentam dispersões características que facilitam a discriminação via covariância.

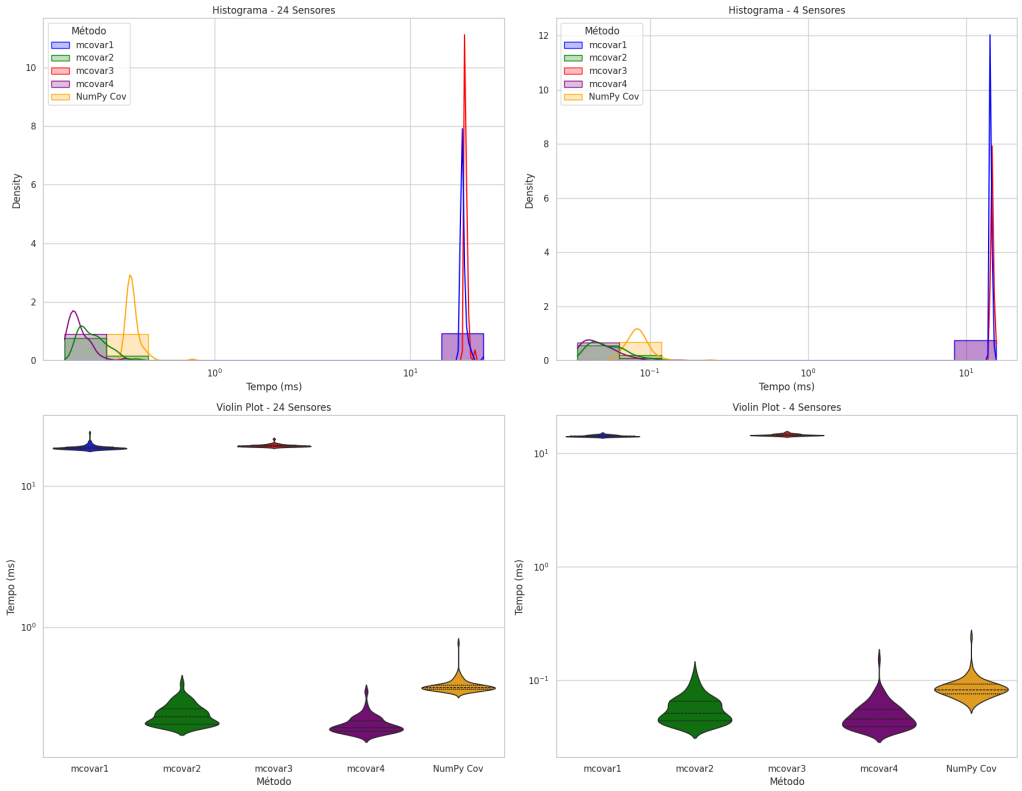

Figura 1. Histogramas e Violin Plots comparativos das leituras dos sensores por classe.

3.2. Análise de Condicionamento e Estabilidade Numérica

Um ponto crítico no cálculo de matrizes de covariância é o seu condicionamento numérico, que determina a sensibilidade da inversão da matriz a pequenas flutuações nos

Tabela 4. Indicadores de Invertibilidade e Estabilidade Numérica.

Classe/Escopo	24 Sensores			4 Sensores		
	Posto	Cond	RCond	Posto	Cond	RCond
Global	24	$2{,}05 \times 10^1$	$4{,}87 \times 10^{-2}$	4	$7{,}57 \times 10^0$	$1{,}32 \times 10^{-1}$
Move-Forward	24	$5{,}35 \times 10^1$	$1{,}87 \times 10^{-2}$	4	$6{,}43 \times 10^1$	$1{,}55 \times 10^{-2}$
Sharp-Right-Turn	24	$3{,}13 \times 10^1$	$3{,}20 \times 10^{-2}$	4	$2{,}15 \times 10^2$	$4{,}65 \times 10^{-3}$
Slight-Left-Turn	24	$1{,}73 \times 10^2$	$5{,}77 \times 10^{-3}$	4	$1{,}11 \times 10^1$	$9{,}00 \times 10^{-2}$
Slight-Right-Turn	24	$1{,}96 \times 10^2$	$5{,}10 \times 10^{-3}$	4	$2{,}19 \times 10^2$	$4{,}57 \times 10^{-3}$

dados. A Tabela **??** detalha os indicadores de invertibilidade (Posto, Cond e RCond) para os cenários global e por classe.

Diferente do esperado para dados redundantes, todas as matrizes apresentaram **posto completo**. O número de condicionamento máximo observado foi da ordem de 10^2, o que é considerado baixo na computação científica. Isso indica que as matrizes são **bem-condicionadas** e estáveis para operações de inversão.

Quanto à Regularização de Tikhonov ($\mathbf{C} + \epsilon\mathbf{I}$), os experimentos confirmaram que, devido ao excelente condicionamento nativo das matrizes, a técnica não produziu mudanças significativas nos resultados ou na capacidade de generalização. Em sistemas mal-condicionados, a regularização é vital para evitar a singularidade ($RCond \rightarrow 0$); contudo, neste dataset, as leituras dos sensores de ultrassom provaram ser suficientemente independentes para manter a integridade numérica das matrizes de covariância originais.

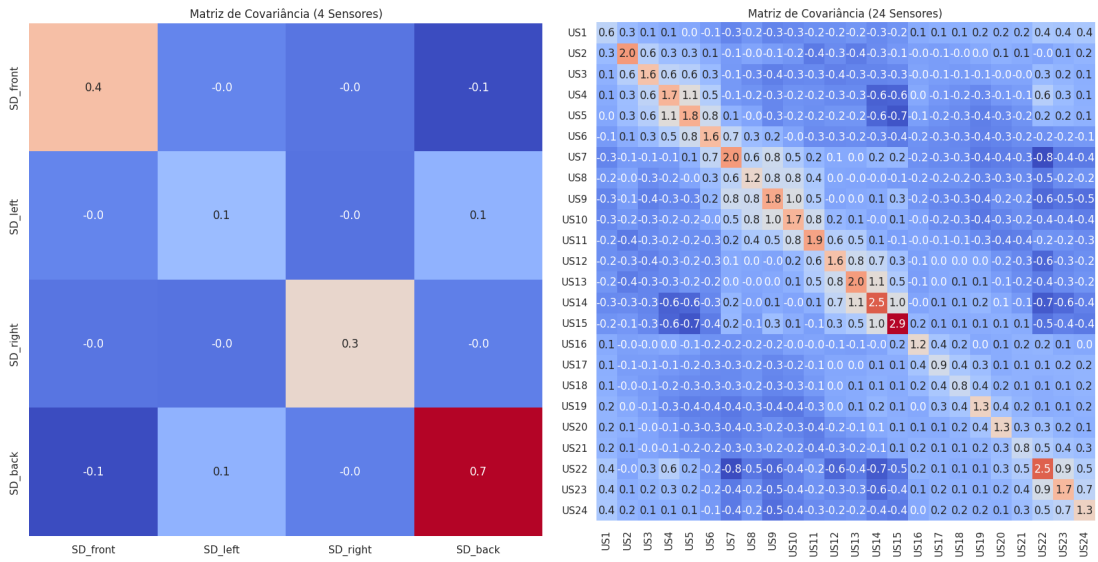

Figura 2. Mapas de calor das matrizes de covariância globais para 4 (esq.) e 24 (dir.) sensores.

4. Conclusão

A comparação sistemática evidenciou que a vetorização é essencial para o processamento de dados em larga escala, com o método `mcovar4` destacando-se pela economia de memória e latência mínima ao evitar a centralização explícita da matriz de dados.

Contrariando hipóteses iniciais de singularidade devido à redundância de sensores, a análise técnica demonstrou que as matrizes de covariância dos datasets do robô *Wall-Following* são inerentemente de posto completo e bem-condicionadas. Este achado é fundamental, pois valida a aplicabilidade direta de modelos probabilísticos quadráticos e transformações lineares sem a necessidade obrigatória de regularização para fins de inversão.

A experimentação com a técnica de Tikhonov, embora não mandatória para este conjunto específico de dados, serviu como uma poderosa ferramenta de análise de sensibilidade, mostrando como a estabilidade numérica pode ser refinada.